实验五：金融场景综合实践

实验报告

班级：_____ 姓名：_____ 学号：_____

实验日期：2026 年 11 月 3 日

实验目标与完成情况

依据《GaussDB 案例》案例五 5.1—5.7 及课程说明第 6—10 页，建立金融业务数据库，完成 JDBC 数据访问、客户与银行卡管理、理财产品申购赎回，以及规定的权限、备份、参数和主备倒换操作。基础任务逐项实现；进阶选择“数据查询与分析”和“数据正确性与可靠性”两个方向。

任务	实际结果
数据库与初始化	finance、UTF8、普通用户 db_dev、同名模式；四张业务表 10/10/4/4 条原始数据
JDBC 与 DAO	官方驱动连接成功；DAO 返回真实影响行数，主键、外键及 CHECK 异常核验通过
业务流程	客户列表与详情、修改、开户、开卡、销卡、销户、产品列表、申购、赎回均完成
管理与运维	DN 倒换前后角色核对、六表快照一致；5 天周末备份策略、手动备份、autoanalyze 及三阶段权限验证完成
两项进阶	三类 CSV 及聚合边界测试；事务回滚、幂等请求、两线程并发竞争及最终数据核对通过

1 实际环境与连接

项目	实际值
实例	gauss-finance-lab；北京四，可用区一/二/七；集中式 1 主 2 备；4vCPU/16GB；40GB 超高 IO
数据库	192.168.0.125:8000；finance；UTF8；模式 finance；开发用户 db_dev
服务端	GaussDB Kernel507.0.0 build d414f3bb，2026-05-31 编译；控制台版本 V2.0-10.0.0
客户端	ECS 私网 192.168.0.126；CentOS7.6.1810，x86_64；OpenJDK11.0.10+9，Eclipse OpenJ9
驱动与工具	官方 gaussdbjdbc.jar；gsql505.1.0 build21af9b4f；源代码以 Java11 编译
网络	vpc-hce、subnet-hce（192.168.0.0/24）、sg-hce；应用通过内网访问数据库



实例已正常运行，页面显示内网地址、端口、VPC 和安全组。

1.1 客户端安装与数据库初始化

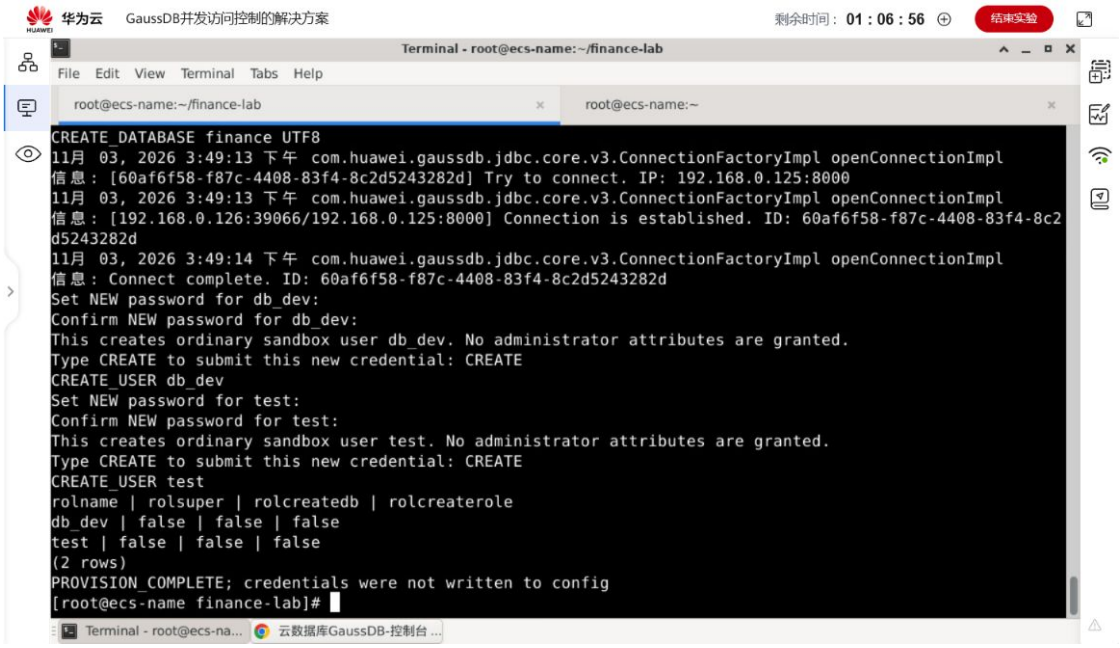
按 ECS 架构选择官方驱动包 `Centralized/Euler2.5_X86_64`。客户端使用明确的安装目录设置 PATH 和动态库路径，运行 Java、gsql 版本检查并编译全部源码。

```
export JAVA_HOME=/root/jdk-11.0.10+9
export PATH="$JAVA_HOME/bin:/opt/bin:$PATH"
export LD_LIBRARY_PATH="/opt/lib:${LD_LIBRARY_PATH:-}"
export GS_KTOOL_FILE_PATH=/opt/gs_ktool_file
bash scripts/compile.sh
bash scripts/run.sh provision 192.168.0.125
```

JDBC 驱动 SHA-256 如下；驱动元数据中的版本字符串保留了厂商构建占位符，因此用官方分发路径和文件摘要准确标识依赖。

```
72ffc8da0de0a41c7c6b6c71f727c78a344a97d06e3d7449bcfeb9c90f580c71
```

```
CREATE DATABASE finance WITH ENCODING='UTF8';
-- db_dev/test are ordinary login users; passwords entered interactively.
CREATE SCHEMA finance AUTHORIZATION db_dev;
GRANT CONNECT ON DATABASE finance TO db_dev,test;
```

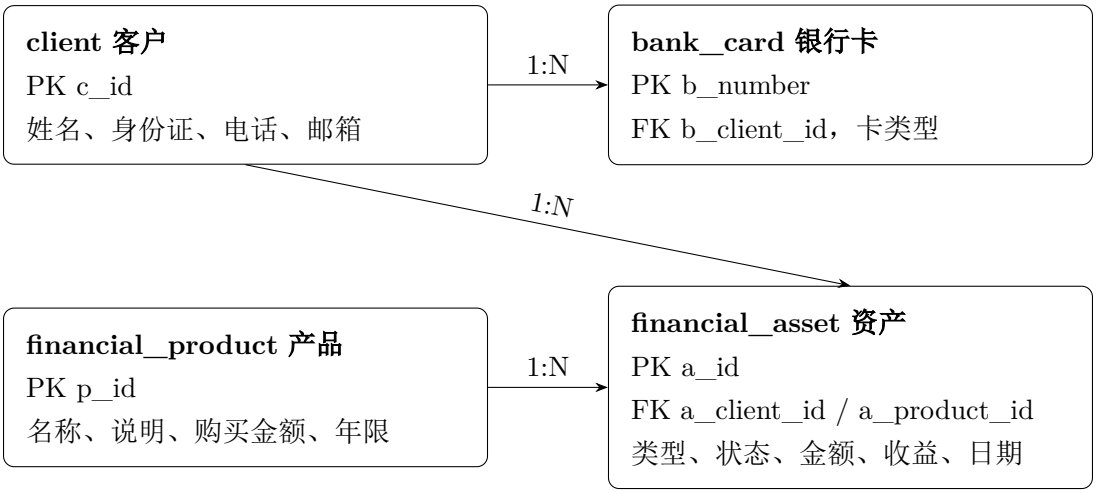


初始化实际输出：数据库创建成功；db_dev 和 test 的管理员、建库与建角色标志均为 false。

2 数据模型、建表与初始化

2.1 实体关系

客户与银行卡是一对多，客户与资产是一对多，理财产品与资产是一对多。资产保留讲义的产品类型、可用/冻结状态、申购金额、收益及购买时间。请求表与事件表用于扩展业务可靠性，每个成功请求对应一条事件。



`command_request.request_id` 为唯一请求号，记录动作、完整参数摘要和结果；`transaction_event.request_id` 同时是主键和指向请求表的外键。事件中保留客户、产品、数量及收益快照，不级联删除历史，使赎回及销户后的交易记录仍可核查。

2.2 完整建表 SQL

下列 SQL 保留讲义四表字段，补充外键、唯一性及取值约束，增加请求/事件两表与四个业务索引。`scripts/01_schema.sql` 由普通用户 db_dev 执行，表和索引归属该用户。

```
CREATE TABLE finance.client (
 c_id INTEGER PRIMARY KEY,
 c_name VARCHAR(100) NOT NULL,
 c_id_card CHAR(20) UNIQUE NOT NULL,
 c_phone CHAR(20) UNIQUE NOT NULL,
 c_mail CHAR(30) UNIQUE
);
CREATE TABLE finance.bank_card (
 b_number CHAR(30) PRIMARY KEY,
 b_type CHAR(20) NOT NULL,
 b_client_id INTEGER NOT NULL REFERENCES finance.client(c_id)
);
CREATE TABLE finance.financial_product (
 p_id INTEGER PRIMARY KEY,
 p_name VARCHAR(100) NOT NULL,
 p_description VARCHAR(1000),
 p_amount INTEGER CHECK (p_amount >= 0),
 p_year INTEGER CHECK (p_year > 0)
);
CREATE TABLE finance.financial_asset (
 a_id INTEGER PRIMARY KEY,
 a_client_id INTEGER NOT NULL REFERENCES finance.client(c_id),
 a_product_id INTEGER NOT NULL REFERENCES finance.financial_product(p_id),
 a_type INTEGER NOT NULL CHECK (a_type=1),
 a_status CHAR(20) NOT NULL CHECK (a_status IN ('可用','冻结')),
 a_quantity INTEGER NOT NULL CHECK (a_quantity>0),
 a_income INTEGER NOT NULL CHECK (a_income>=0),
 a_purchase_time DATE NOT NULL
);
CREATE TABLE finance.command_request (
 request_id VARCHAR(80) PRIMARY KEY,
 action VARCHAR(16) NOT NULL CHECK (action IN ('BUY','REDEEM')),
 payload_hash CHAR(64) NOT NULL,
 result_id INTEGER,
 result_code VARCHAR(16) NOT NULL CHECK (result_code IN ('PENDING','SUCCESS'))
);
CREATE TABLE finance.transaction_event (
 request_id VARCHAR(80) PRIMARY KEY REFERENCES finance.command_request(request_id),
 action VARCHAR(16) NOT NULL,
 asset_id INTEGER NOT NULL,
 client_id INTEGER NOT NULL,
 product_id INTEGER NOT NULL,
 quantity INTEGER NOT NULL CHECK (quantity>0),
 income INTEGER NOT NULL CHECK (income>=0),
 event_time TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_card_client ON finance.bank_card(b_client_id);
CREATE INDEX idx_asset_client ON finance.financial_asset(a_client_id);
CREATE INDEX idx_asset_product ON finance.financial_asset(a_product_id);
CREATE INDEX idx_asset_purchase ON finance.financial_asset(a_purchase_time);
```

2.3 原始数据与口径

scripts/02_seed.sql 包含讲义原始 28 条 INSERT：10 客户、10 银行卡、4 产品、4 资产。原始资产如下。

资产	客户	产品	状态	申购金额	收益	购买日期
1	1	1	可用	4	8001	2018-07-01
2	2	2	可用	3	8002	2018-07-02
3	3	3	可用	2	8003	2018-07-03
4	4	4	冻结	1	8004	2018-07-04

申购金额合计 10，收益合计 32010；这些是讲义示例字段值，不据此推导投资回报率。p_amount 保留产品购买金额口径，不额外解释为最低申购门槛。仅实现讲义理财产品类型 1，未增加保险或基金业务。初始化遇到已有表或非空数据时停止，不覆盖已有内容。种子插入与行数检查在同一事务中完成。

```
INSERT INTO finance.financial_asset(a_id, a_client_id, a_product_id, a_type, a_status, a_quantity, a_income,
    a_purchase_time) VALUES (1, 1, 1, 1, '可用', 4, 8001, TO_DATE('2018-07-01','YYYY-MM-DD'));
INSERT INTO finance.financial_asset(a_id, a_client_id, a_product_id, a_type, a_status, a_quantity, a_income,
    a_purchase_time) VALUES (2, 2, 2, 1, '可用', 3, 8002, TO_DATE('2018-07-02','YYYY-MM-DD'));
INSERT INTO finance.financial_asset(a_id, a_client_id, a_product_id, a_type, a_status, a_quantity, a_income,
    a_purchase_time) VALUES (3, 3, 3, 1, '可用', 2, 8003, TO_DATE('2018-07-03','YYYY-MM-DD'));
INSERT INTO finance.financial_asset(a_id, a_client_id, a_product_id, a_type, a_status, a_quantity, a_income,
    a_purchase_time) VALUES (4, 4, 4, 1, '冻结', 1, 8004, TO_DATE('2018-07-04','YYYY-MM-DD'));
```

3 JDBC、DAO 与业务流程

3.1 分层与返回值检查

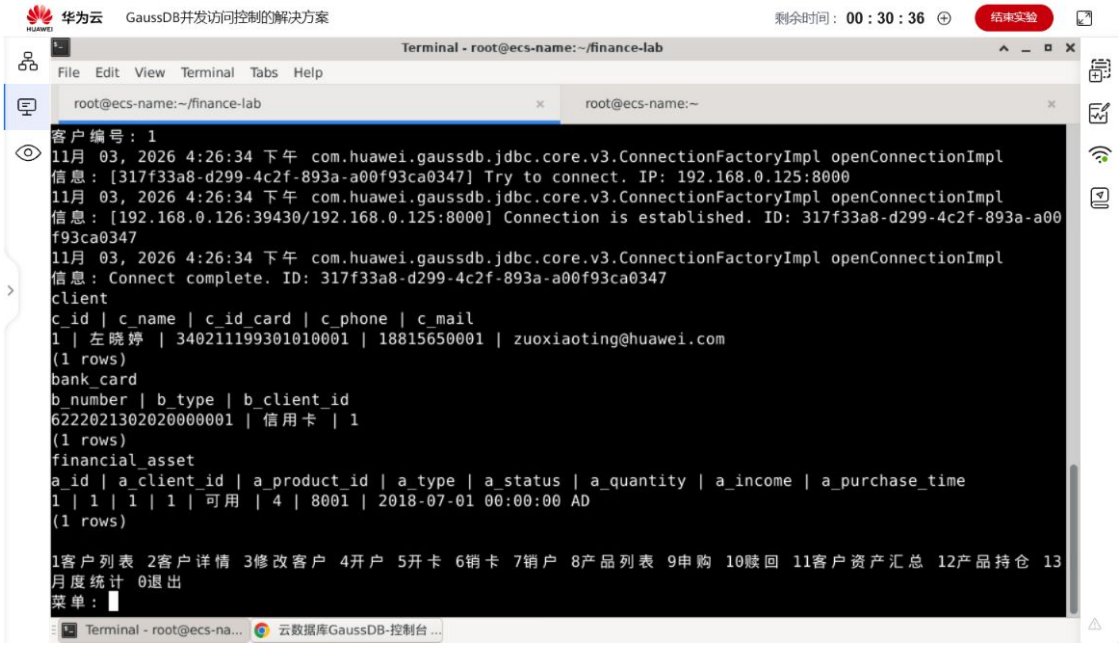
DBUtils.java 集中处理配置、独立连接、PreparedStatement 和结果转换；四个 DAO 分别处理客户、银行卡、产品、资产。Service.java 负责规则和事务，FlowControl.java 组织菜单，launch.java 提供入口。配置只包含驱动类、URL 和用户；密码通过控制台交互取得，不写入配置。SQL 参数使用绑定变量，连接、语句及结果集使用 try-with-resources 关闭；语句设置 45 秒超时。DAO 直接返回真实影响行数，业务层要求应修改一行的操作实际影响一行，否则抛出 P0002，避免 0 行也显示成功。

```
private static void one(int affected) throws SQLException {
    if (affected != 1)
        throw new SQLException("Expected one affected row; actual="+affected,"P0002");
}
```

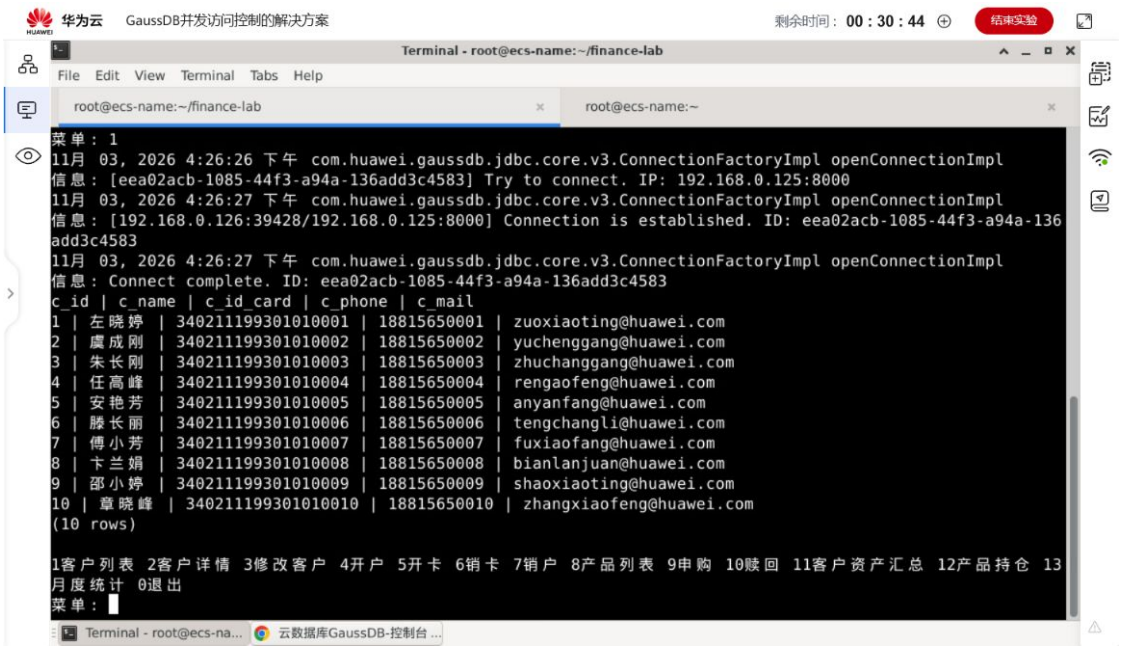
3.2 十项基础菜单

菜单	Service 方法	结果与关键规则
1/2 查询	clients / detail	列表及客户、银行卡、资产关联详情；详情在一致事务中读取
3 修改	modify	校验姓名、11 位电话和邮箱；不存在客户拒绝，实际影响 1 行才成功
4 开户	openAccount	检查正整数 ID 及 18 位身份证格式；唯一约束防止重复身份、电话
5/6 开卡销卡	openCard / closeCard	卡号 16—30 位；卡类型限制；销卡必须匹配所属客户
7 销户	closeAccount	锁定客户后检查银行卡和资产，仍有任一项时拒绝销户
8 产品	products	返回四条原始产品及说明、购买金额、年限
9 申购	buy	正金额、存在客户和产品；持仓、请求及事件同一事务提交
10 赎回	redeem	持仓必须属于客户且可用；删除持仓并保留赎回事件

实际业务测试采用独立测试客户 910001，依次开户、修改、开卡、申购、赎回、销卡、销户。结束后该客户和临时持仓已清除，原始业务表保持 10/10/4/4，交易历史仍保留 4 条。`testDAO.java` 验证列表、主键、无记录、增改删和 0 行返回，以及 23505 主键/唯一约束、23503 外键、23514 CHECK 约束；DAO 测试最终回滚。



实际菜单 2：客户 1 的基本信息、银行卡及资产详情由同一业务入口返回。



实际菜单 1：查询讲义的 10 名客户。菜单程序还实际执行产品列表及 11—13 三类报表后正常退出。

4 进阶一：数据查询与分析

4.1 目标与实现

增加客户资产汇总、产品持仓、按时间区间的月度统计，菜单 11—13 可直接查询，`Reports.java` 可导出 UTF8 CSV。卡和资产分别预聚合后再 LEFT JOIN 到客户，避免一名客户同时有多卡、多资产时发生行数相乘；LEFT JOIN 及 COALESCE 保留零持仓客户。冻结与可用金额分别统计。

```
WITH cards AS (
 SELECT b_client_id,COUNT(*) AS card_count
 FROM finance.bank_card GROUP BY b_client_id
), assets AS (
 SELECT a_client_id,COUNT(*) AS asset_count,
 SUM(a_quantity) AS quantity_total,SUM(a_income) AS income_total,
 SUM(CASE WHEN a_status='可用' THEN a_quantity ELSE 0 END) AS available_quantity,
 SUM(CASE WHEN a_status='冻结' THEN a_quantity ELSE 0 END) AS frozen_quantity
 FROM finance.financial_asset GROUP BY a_client_id
)
SELECT c.c_id,c.c_name,COALESCE(b.card_count,0) AS card_count,
 COALESCE(a.asset_count,0) AS asset_count,
 COALESCE(a.quantity_total,0) AS quantity_total,
 COALESCE(a.income_total,0) AS income_total,
 COALESCE(a.available_quantity,0) AS available_quantity,
 COALESCE(a.frozen_quantity,0) AS frozen_quantity
FROM finance.client c LEFT JOIN cards b ON b.b_client_id=c.c_id
LEFT JOIN assets a ON a.a_client_id=c.c_id
ORDER BY quantity_total DESC,c.c_id;
```

```
SELECT p.p_id,p.p_name,COUNT(a.a_id) AS asset_count,
```

```
 COUNT(DISTINCT a.a_client_id) AS client_count,
 COALESCE(SUM(a.a_quantity),0) AS quantity_total,
 COALESCE(SUM(a.a_income),0) AS income_total,
 COALESCE(SUM(CASE WHEN a.a_status='冻结' THEN a.a_quantity ELSE 0 END),0) AS frozen_quantity
FROM finance.financial_product p
LEFT JOIN finance.financial_asset a ON a.a_product_id=p.p_id
GROUP BY p.p_id,p.p_name ORDER BY p.p_id;
```

月度查询使用起日包含、止日不含的半开区间，按显式 YYYY-MM 格式分组。该报表统计**当前持仓的申购日期**，不冒充包含已赎回持仓的完整历史流水。

```
SELECT TO_CHAR(a_purchase_time,'YYYY-MM') AS purchase_month,
 COUNT(*) AS purchase_count,COUNT(DISTINCT a_client_id) AS client_count,
 SUM(a_quantity) AS quantity_total,SUM(a_income) AS income_total
FROM finance.financial_asset
WHERE a_purchase_time>=? AND a_purchase_time<?
GROUP BY TO_CHAR(a_purchase_time,'YYYY-MM')
ORDER BY purchase_month;
```

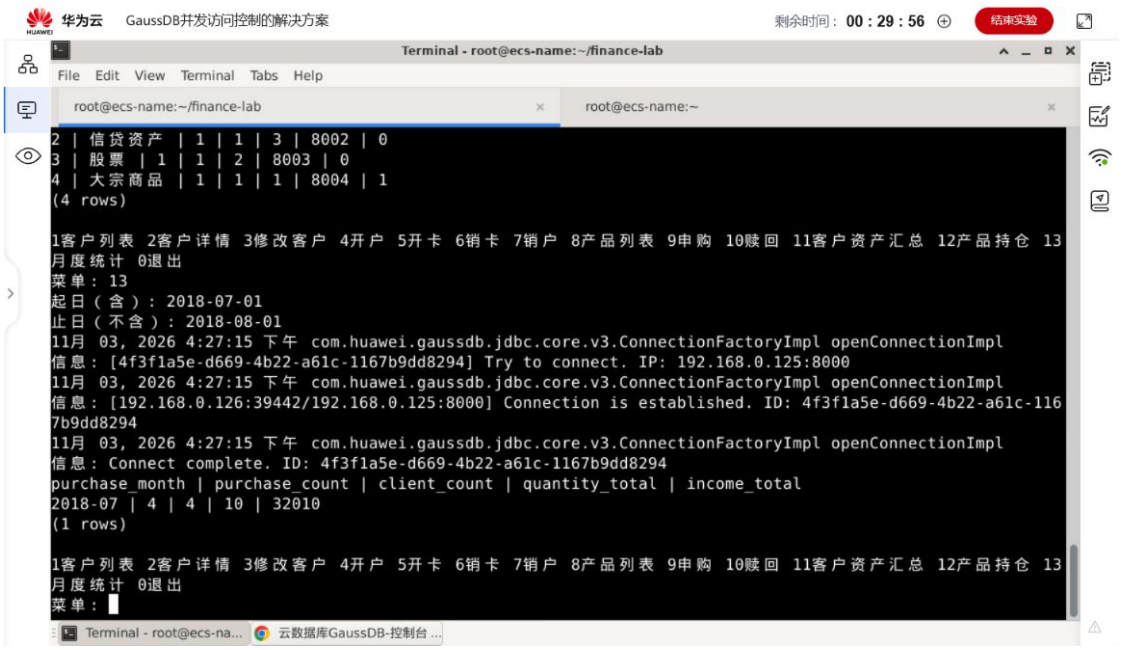
4.2 独立期望值与边界核验

在回滚事务内给客户 1 增加一张卡及两条资产：2018-07-31 可用 100、收益 7；2018-08-01 冻结 200、收益 9。其原始资产金额 4、收益 8001，因此预期 2 卡、3 资产、金额 304、收益 8017、可用 104、冻结 200。若直接连接卡和资产求和，金额会变为 608；实际预聚合结果为 304。客户 10 无持仓，仍出现在结果中且金额 0；产品 1 的不同客户数为 1，资产数 3；7 月区间含 7 月 31 日、排除 8 月 1 日，预期 5 笔、金额 110、收益 32017；8 月区间预期 1 笔、金额 200。全部通过，测试数据回滚。

```
PASS 1 Ordinary development user db_dev connected
PASS 2 Report preaggregation: two cards/three assets, amount 304 not 608
PASS 3 Report preserves zero-holding client
PASS 4 Product distinct holder count and amount
PASS 5 July includes July 31, excludes August 1; amount 110
PASS 6 August includes lower boundary once
```

原始种子数据的月度 CSV 如下，客户和产品报表分别为 10 行和 4 行。

```
purchase_month,purchase_count,client_count,quantity_total,income_total
"2018-07","4","4","10","32010"
```



实际菜单 13：2018-07-01 至 2018-08-01，返回 4 笔、4 客户、金额 10、收益 32010。

5 进阶二：数据正确性与可靠性

5.1 跨表事务、行锁与失败回滚

申购/赎回将持仓变化、唯一请求结果及事件记录放入同一 READ COMMITTED 事务。统一先锁客户行，再读取或锁定持仓，使开卡、销卡、申购、赎回、销户对同一客户遵守相同锁顺序。销户检查与删除保持在同一事务，避免检查后又插入新卡或新资产的竞争。

```
c.setTransactionIsolation(Connection.TRANSACTION_READ_COMMITTED);
c.setAutoCommit(false);
try {
    T result = work.run(c);
    c.commit();
    return result;
} catch (Exception e) {
    c.rollback();
    throw e;
}
```

测试在资产插入后、事件写入前主动抛出 P0001，独立连接确认资产、请求、事件均为 0 条，无部分提交。冻结持仓赎回、未知客户/产品、错误所属者、有卡或有资产销户均拒绝，并核对失败请求未残留。

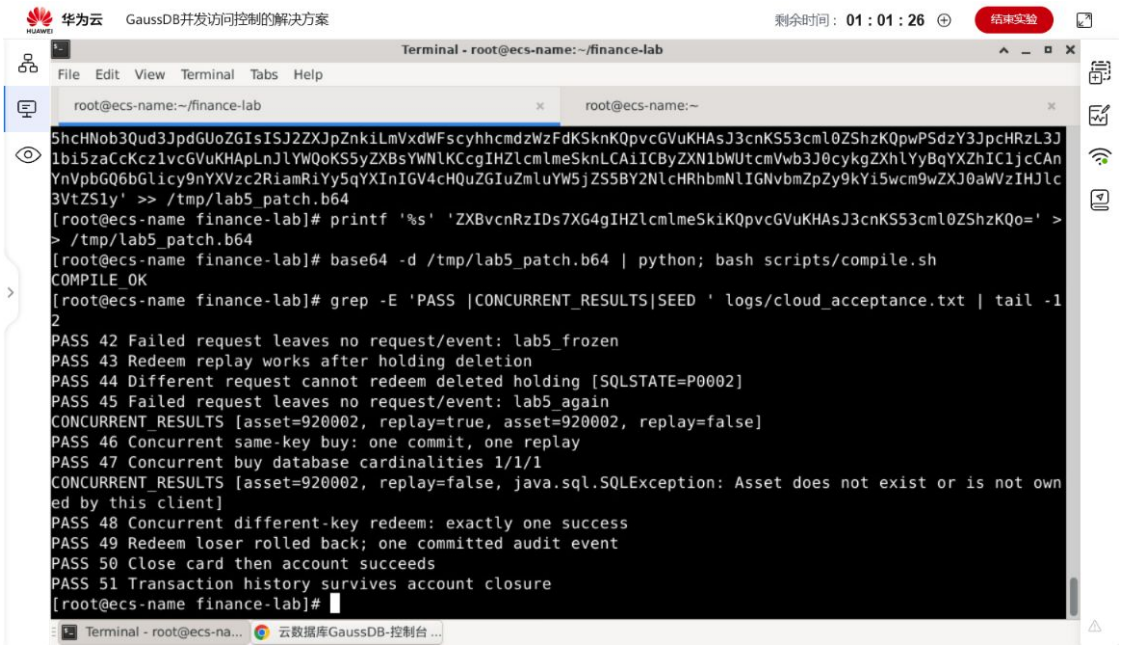
5.2 幂等请求与参数冲突

请求号是请求表主键，参数摘要包含动作、资产、客户、产品、金额和日期。首次请求插入 PENDING；业务与事件写入成功后更新为 SUCCESS 再提交。相同请求号、相同完整

参数返回原资产编号且 replay=true；相同请求号但金额、日期或动作不同则拒绝。仅在请求插入发生 23505 时进入重放分支；其他业务主键错误正常抛出。冲突后先 rollback，再读取已提交请求，校验动作、摘要、状态，避免将任意唯一约束错误误当成功。赎回后即使持仓已删除，仍能依据请求记录重放原结果。

5.3 真实并发验证

`ScenarioTest.java` 使用两个线程、CountDownLatch 起跑门及各自独立 JDBC 连接。两线程以相同请求号申购，实测一笔提交、一笔重放，资产/请求/事件各 1 条。随后两线程以不同请求号赎回同一资产，实测一笔成功，一笔 P0002 失败；失败请求整体回滚，仅保留一个赎回事件。



并发实际结果：相同请求申购只生成一笔；同一持仓赎回只成功一次；销卡、销户后仍保留 4 条交易历史。

5.4 进阶评价对应

评价维度	实现与可核验材料
技术深度（8 分）	跨表事务、唯一约束、参数摘要、行锁顺序、并发竞争；多表预聚合及半开日期边界。对应 Service、Reports、三份报表 SQL。
改进价值（6 分）	防止重复提交、失败部分写入、销户检查竞争和关联汇总重复计数；修复 0 行仍报成功及异常处理不清的问题。
流程与验证（6 分）	基础业务完整闭环；正常、参数非法、SQL 约束、冻结、故障注入、重放冲突、真实并发、报表独立期望值、gsql 交叉核对及切换后六表一致性。

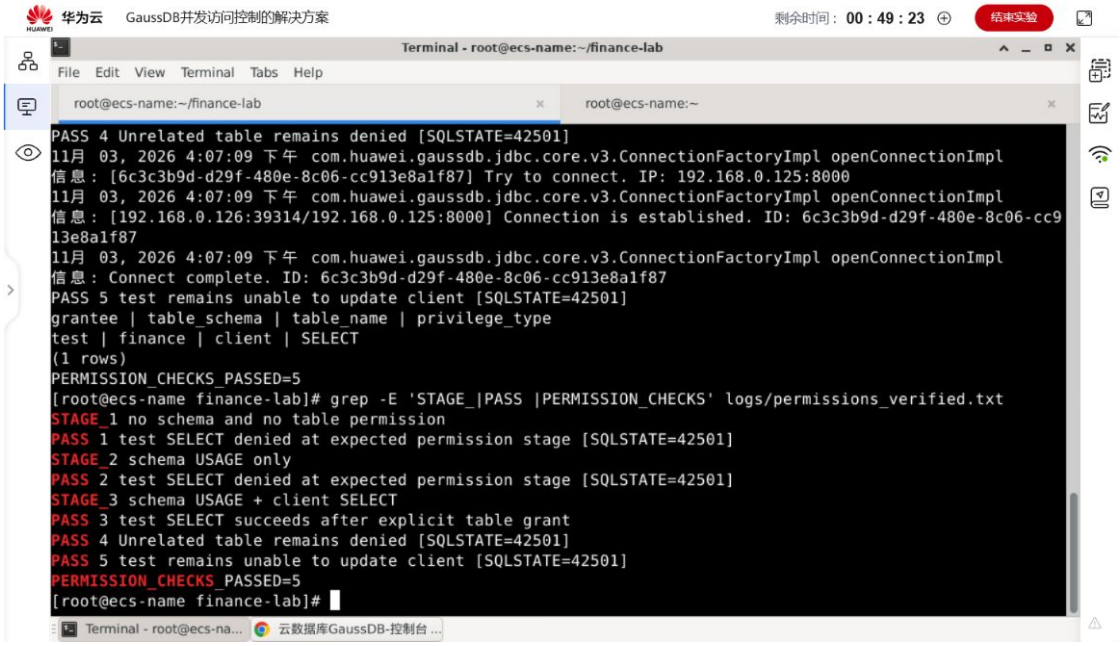
上述为评价依据映射，最终得分由课程按实现质量综合评定；两个方向不简单各计 10 分。

6 管理与运维

6.1 普通用户三阶段权限

test 为普通用户，先验证无模式权限时 SELECT 被 42501 拒绝；db_dev 授予模式 USAGE 后，仍无表读取权限；再仅授予 client 表 SELECT，查询 10 条客户成功。查询 bank_card 和 UPDATE client 仍以 42501 被拒绝。

```
GRANT USAGE ON SCHEMA finance TO test;
GRANT SELECT ON finance.client TO test;
SELECT grantee,table_schema,table_name,privilege_type
FROM information_schema.role_table_grants WHERE grantee='test';
```



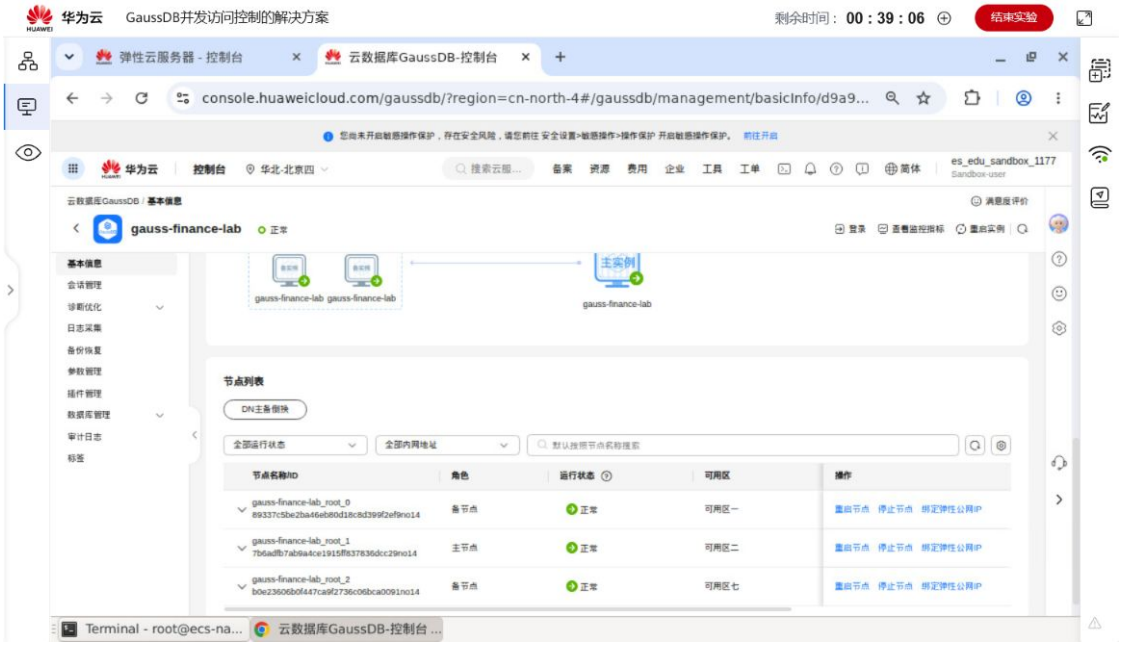
三阶段 5 项检查全通过；test 最终只具有 finance.client 的 SELECT 权限。

6.2 DN 主备倒换与数据一致性

切换前 root_0 为主节点（可用区一），root_1/2 为备节点（可用区二/七）。控制台选择 root_1 执行 DN 倒换；完成后 root_1 为主、root_0 为备，三节点正常。重新建立连接，使用原浮动地址 192.168.0.125 访问 finance。



切换前的三节点角色。

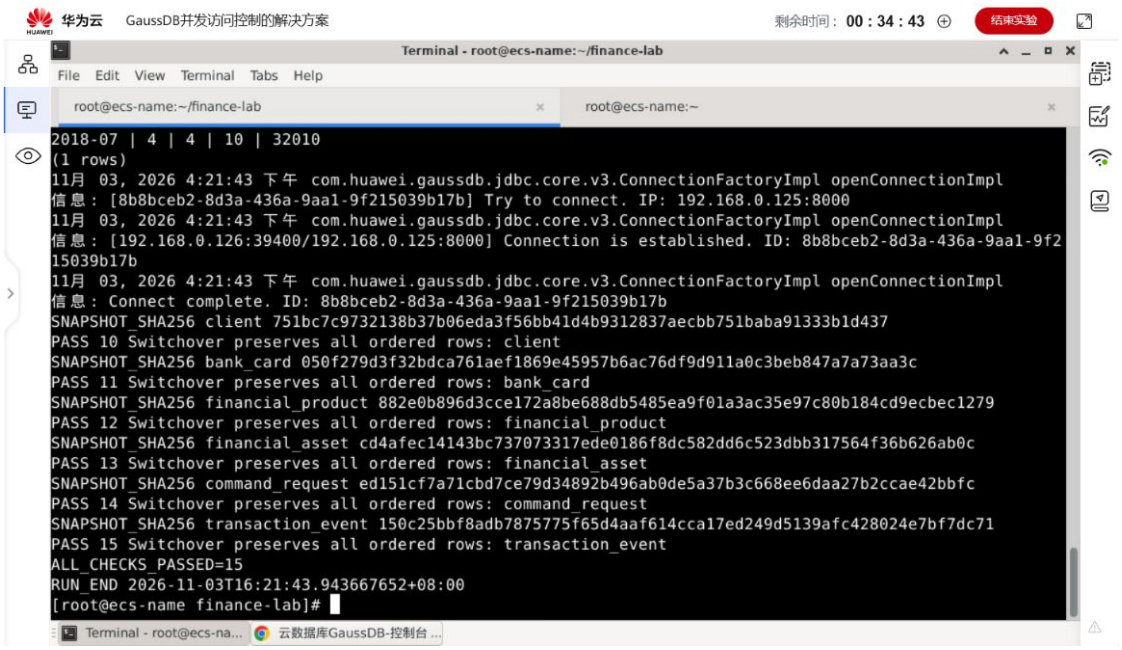


切换后 root_1 为主、root_0 为备，实例和节点均正常。

Snapshot 在 REPEATABLE READ 事务内按各表主键排序导出完整 CSV；切换后再次导出，与切换前文件逐字节比较。以下六表摘要均一致，覆盖原始字段、请求结果及交易事件时间。

```
bank_card
050f279d3f32bdca761aef1869e45957b6ac76df9d911a0c3beb847a7a73aa3c
identical=true
client
751bc7c9732138b37b06eda3f56bb41d4b9312837aecbb751baba91333b1d437
identical=true
command_request
ed151cf7a71cbd7ce79d34892b496ab0de5a37b3c668ee6daa27b2ccae42bbfc
identical=true
```

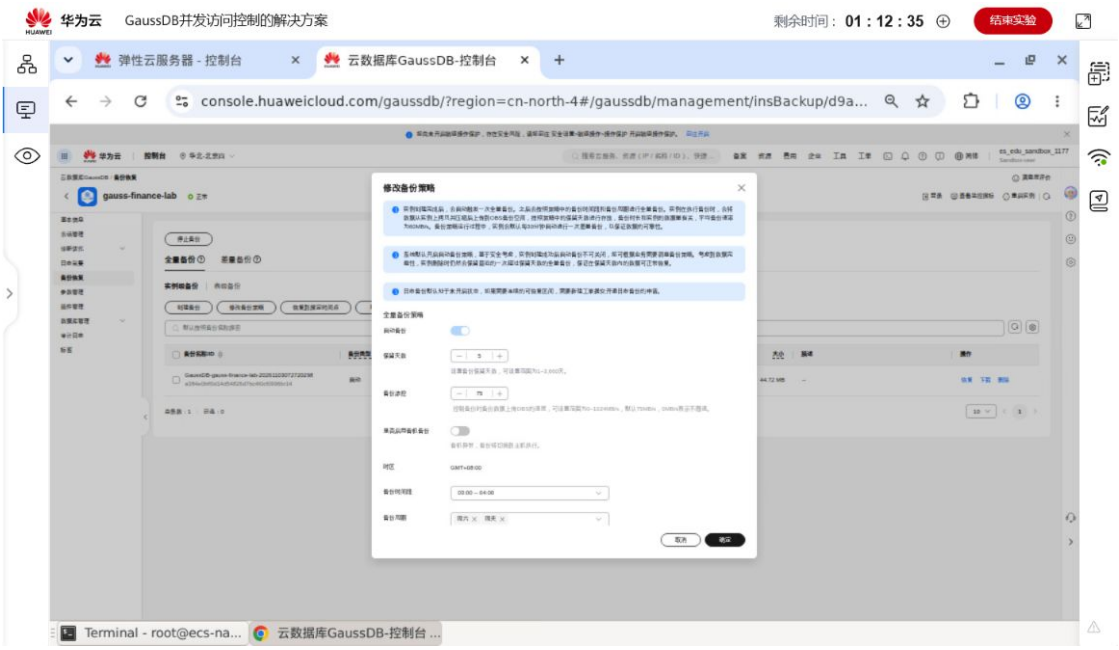
```
financial_asset
cd4afec14143bc737073317ede0186f8dc582dd6c523dbb317564f36b626ab0c
identical=true
financial_product
882e0b896d3cce172a8be688db5485ea9f01a3ac35e97c80b184cd9ecbec1279
identical=true
transaction_event
150c25bbf8adb7875775f65d4aaf614cca17ed249d5139afc428024e7bf7dc71
identical=true
```



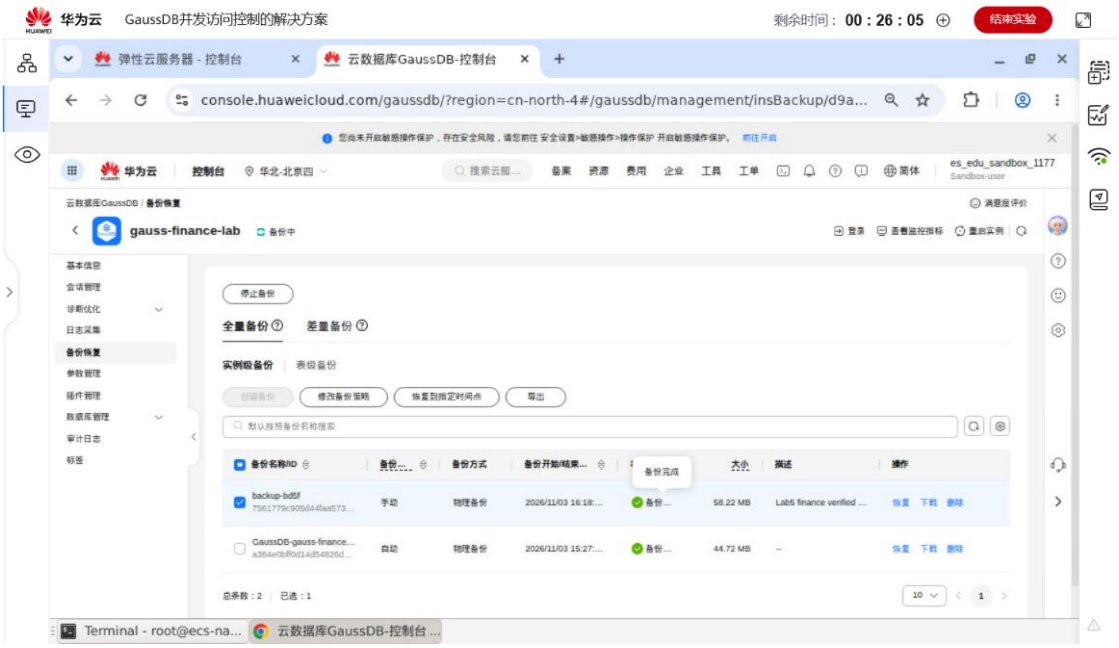
切换后 15 项核验通过，包含六表完整有序记录比对。

6.3 备份策略与手动备份

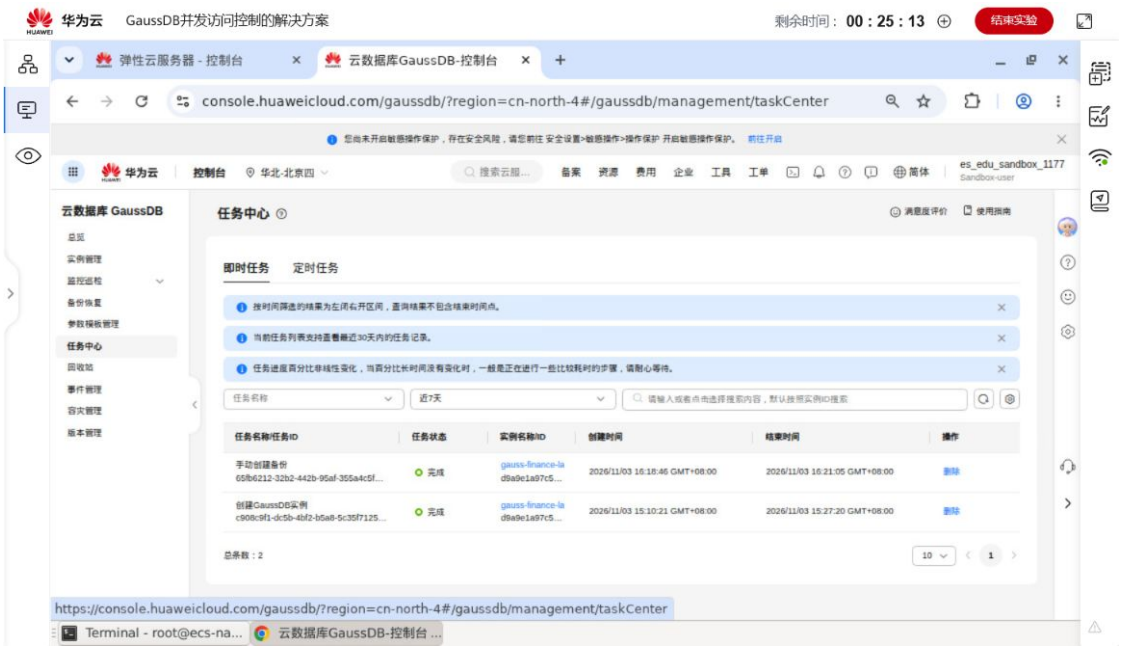
自动备份策略保存后重新打开核对：保留 5 天，每周六和周日 03:00—04:00（GMT+08:00）。手动备份 backup-bd5f 在 2026-11-03 16:18:46 开始，16:21:05 完成，大小 58.22MB。备份列表显示“备份完成”，任务中心显示“完成”。



保存后重新读取：保留天数 5，周期为周六和周日。



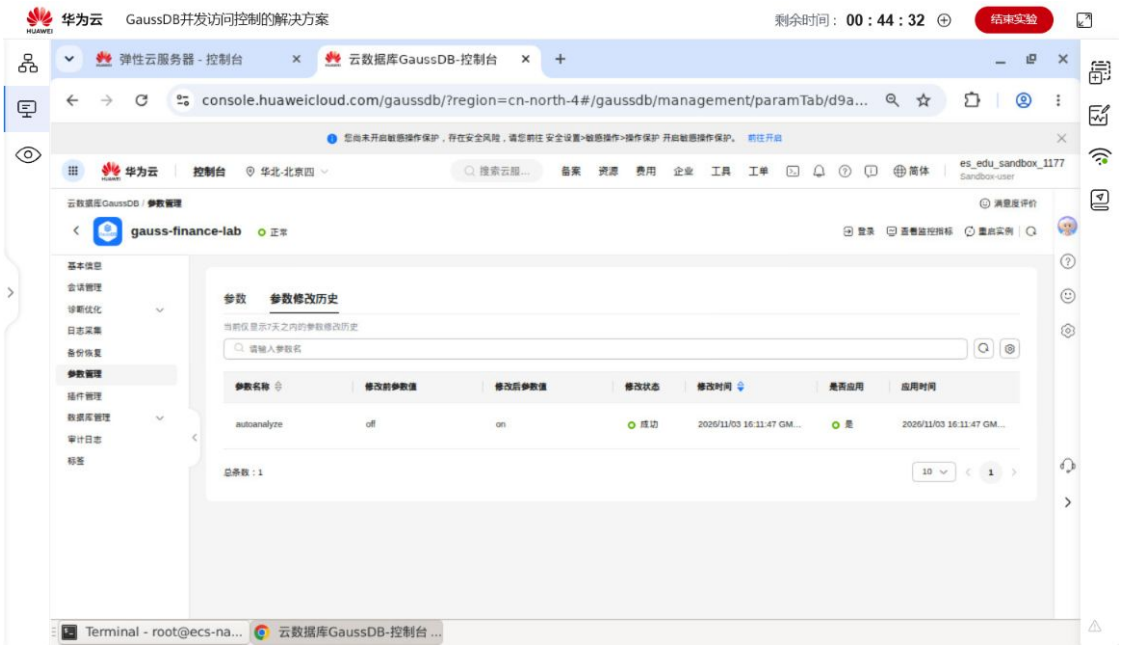
手动备份 backup-bd5f 完成；状态提示及 58.22MB 大小可核对。



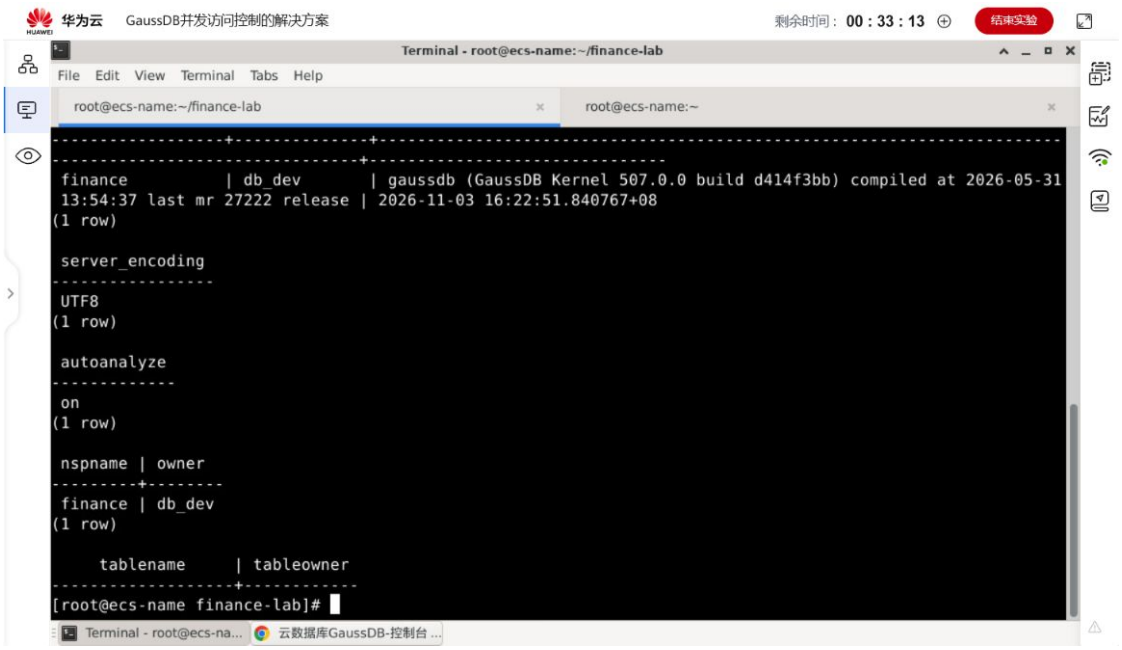
任务中心记录手动备份完成及起止时间。

6.4 autoanalyze 参数及系统目录

控制台将 autoanalyze 从 off 设为 on；修改历史显示成功且已应用。独立 gsql 执行 SHOW autoanalyze 返回 on，SHOW server_encoding 返回 UTF8；模式归属及所有表归属均核对为 db_dev。



参数修改历史：off 变为 on，状态成功、已应用。



独立 gsql 核对：UTF8、autoanalyze=on、finance 模式归属 db_dev。

```
SELECT tablename,tableowner FROM pg_tables
WHERE schemaname='finance' ORDER BY tablename;
SELECT tablename,indexname,indexdef FROM pg_indexes
WHERE schemaname='finance' ORDER BY tablename,indexname;
```

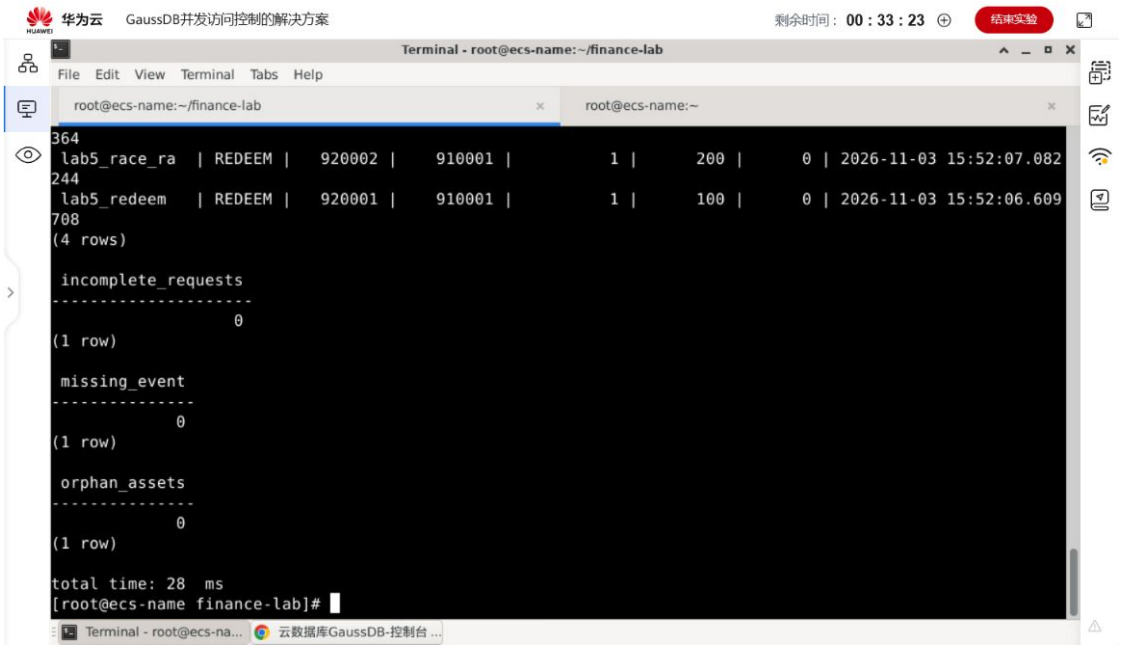
六张表均有主键索引；client 身份证、电话、邮箱具有唯一索引；另外创建银行卡客户、资产客户、产品和购买日期四个业务索引。查询来自 db_dev 普通用户的新连接，结果保存于独立 gsql 日志。

6.5 独立交叉核对

scripts/03_independent_verify.sql 使用 ON_ERROR_STOP，任一 SQL 失败立即停止。除版本、编码、参数和对象归属外，还检查六表行数、原始持仓、金额 10 与收益 32010、4 条成功请求和 4 条事件；未完成请求、缺失事件、孤立持仓均为 0。

```
\set ON_ERROR_STOP on
\pset pager off
\conninfo
SELECT current_database(),current_user,version(),current_timestamp;
SHOW server_encoding;
SHOW autoanalyze;
SELECT nspname,pg_get_userbyid(nspowner) AS owner FROM pg_namespace WHERE nspname='finance';
SELECT tablename,tableowner FROM pg_tables WHERE schemaname='finance' ORDER BY tablename;
SELECT tablename,indexname,indexdef FROM pg_indexes WHERE schemaname='finance' ORDER BY tablename,indexname;
SELECT 'client' AS table_name,count(*) AS rows FROM finance.client UNION ALL
SELECT 'bank_card',count(*) FROM finance.bank_card UNION ALL
SELECT 'financial_product',count(*) FROM finance.financial_product UNION ALL
SELECT 'financial_asset',count(*) FROM finance.financial_asset UNION ALL
SELECT 'command_request',count(*) FROM finance.command_request UNION ALL
SELECT 'transaction_event',count(*) FROM finance.transaction_event;
```

```
SELECT a_id,a_client_id,a_product_id,a_type,a_status,a_quantity,a_income,a_purchase_time FROM finance.
    financial_asset ORDER BY a_id;
SELECT sum(a_quantity) AS total_quantity,sum(a_income) AS total_income FROM finance.financial_asset;
SELECT request_id,action,result_id,result_code FROM finance.command_request ORDER BY request_id;
SELECT * FROM finance.transaction_event ORDER BY request_id;
SELECT count(*) AS incomplete_requests FROM finance.command_request WHERE result_code<>'SUCCESS';
SELECT count(*) AS missing_event FROM finance.command_request r LEFT JOIN finance.transaction_event e ON e.
    request_id=r.request_id WHERE e.request_id IS NULL;
SELECT count(*) AS orphan_assets FROM finance.financial_asset a LEFT JOIN finance.client c ON a.a_client_id=c.
    c_id LEFT JOIN finance.financial_product p ON a.a_product_id=p.p_id WHERE c.c_id IS NULL OR p.p_id IS NULL
    ;
```

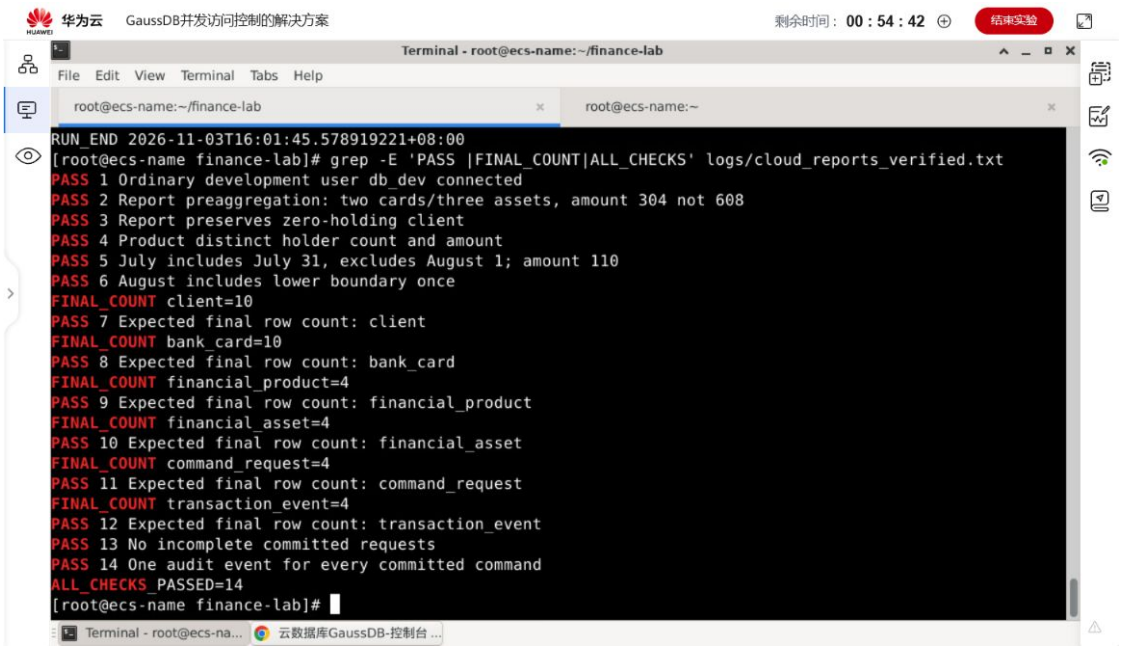


独立 gsql 的完整性检查：三个异常计数均为 0。

7 运行结果、问题处理与复现

7.1 实际验收结果

首轮云端日志 `logs/cloud/cloud_acceptance.txt` 中前 51 项断言通过，覆盖 DAO、业务正常/异常、事务、幂等及并发，随后在报表测试的日期字面量处停止。修正日期绑定和月份表达式后，`cloud_reports_verified.txt` 续测 14 项全部通过，导出三类 CSV 及切换前快照。权限验证 5 项通过；切换后 15 项通过；独立 gsql 脚本完整执行。上述是不同阶段的实际检查数，不将重复项目累加为独立测试数。



最终行数为 10/10/4/4/4/4；没有未完成请求，每个成功请求对应一个事件。

7.2 日期兼容问题及修正

当前实例对日期字面量和日期转文本有特定格式要求：DATE '2018-07-31' 触发月份解析错误；日期直接 CAST 为文本再截前 7 位不能可靠得到 YYYY-MM。测试数据改用 PreparedStatement 绑定 java.sql.Date，月度分组改为 TO_CHAR(a_purchase_time,'YYYY-MM')。修正后 7 月 31 日与 8 月 1 日边界及原始 7 月统计全部通过。失败阶段保留原始日志，报表夹具在 finally 中回滚，未污染种子数据。

7.3 编译、执行顺序

依赖、配置与运行命令在项目 README 中完整列明。新环境按下列顺序执行：

```
bash scripts/setup.sh
bash scripts/run.sh provision <sandbox-private-ip>
bash scripts/run.sh acceptance
bash scripts/run.sh launch
bash scripts/run.sh permissions <sandbox-private-ip>
# After the DN switchover has completed:
bash scripts/run.sh verify
gsql -h <sandbox-private-ip> -p 8000 -d finance -U db_dev \
  -f scripts/03_independent_verify.sql
```

初始化只适用于空数据库；完整业务验收要求保留的测试 ID 和 lab5_ 请求号尚未被使用。已有验收结果时使用只读 verify，不盲目重做建表或删除现有数据。权限三阶段程序要求 test 初始未授权。CSV 报表可通过 reports 动作重新生成；`resume-reports` 仅用于本次修正报表测试后的续测，不重复执行已完成的业务变更。

8 实验总结

本实验将数据库建模、JDBC 访问、业务规则与运维验证串联为可复现流程。普通用户完成四表业务及扩展表操作，数据库约束与应用层参数校验共同防止无效数据。申购和赎回通过事务、请求号和事件记录保持结果一致；并发测试证明唯一请求和行锁能约束重复交易，失败路径没有部分提交。

报表设计的关键是先明确粒度与日期口径：银行卡和持仓分别汇总可避免金额重复，零持仓客户不能被内连接丢失，月度边界采用半开区间并显式格式化。主备倒换后的全字段快照与独立 gsql 核对，为程序断言提供了交叉证据。后续若扩展为实际业务系统，还需加入金额精度与币种、真实收益计算、连接池及认证界面，并按业务权限划分角色；本次实现遵循教学示例的整数金额和理财产品范围。

资料与代码来源

课程《数据库课程期末大作业说明》第 6—12 页；《GaussDB 案例》案例五 5.1—5.7 提供业务字段、种子数据、基础流程及运维步骤。项目据此实现 DAO、Service 和菜单，并增加报表、事务与验证。官方 GaussDB 驱动分发包及华为云教学资源用于 JDBC/gsql 与 JDK 依赖，下载地址记录于 `scripts/setup.sh`。控制台、SQL 和程序结果以本报告截图及配套实际日志为依据。